

Cashflow2.0 - Performance Deep Dive & Optimization Opportunities

Full audit of the frontend and backend, covering what's already implemented (accurately, based on reading the actual code) and every potential optimization found: how it would work, what it would take, whether it's actually feasible, and how much it would likely help. Written in response to reports that login/signup take noticeably long, and that automatic categorization stalls for a while even after reaching 100%.

1. Infrastructure / deployment

What's implemented: connection pooling (`SimpleConnectionPool`, 1-10 conns) between the app and Supabase's session pooler; a scheduled GitHub Action that pings Supabase's database every 3 days specifically to prevent its 7-day free-tier auto-pause (queries a real table, not just the API gateway, since that's what Supabase's pause detection actually checks); a separate nightly backup workflow.

What's missing / the gap: nothing pings the **Render web service itself**. If it's on Render's free tier, the process spins down after ~15 minutes idle and cold-starts in 30-60s on the next request - every dependency (gunicorn boot, Flask import graph, DB pool init) happens synchronously before the first response. This is, with high confidence, the actual cause of "login and signup take a sec": it's not that login is slow, it's that the **first** request after any idle period is slow, and login is very often that first request.

- **Fix:** add a scheduled ping (every ~10 min) to a lightweight `/health` route, same pattern as the existing DB keep-alive workflow.
 - **Feasibility:** trivial - one new GitHub Actions workflow, no app code changes.
 - **Impact:** large and immediate for the specific "takes a sec" complaint, but only for cold starts; it does nothing for genuinely warm-request latency (see sections below for that).
 - **Caveat:** if you're intentionally on Render's free tier to avoid cost, keeping it artificially warm 24/7 defeats some of that tier's point - worth deciding if that trade-off is wanted, or whether upgrading to a paid instance (which doesn't sleep) is the more honest fix.
-

2. Backend - authentication & permission overhead

What's implemented: JWT auth via flask-jwt-extended with real server-side revocation (`revoked_tokens` table, checked via `token_in_blocklist_loader`), short-lived access tokens (24h) + refresh tokens (30d), a real role/permission system (roles -> role_permissions -> per-user overrides) replacing an old hardcoded username check.

What digging into it found: every single authenticated request pays for this correctness, sequentially:

1. **check_if_token_revoked - 1 DB round trip** (SELECT 1 FROM revoked_tokens WHERE jti = %s), on literally every @jwt_required() route.
2. Any route with @require_permission(...) then runs get_user_role_and_permissions, which is **3 more sequential DB round trips**: role lookup -> role_permissions join -> user_permission_overrides join - run in full even for an owner, whose actual permission check is a one-line short-circuit *after* all three queries already ran.

So a single gated action (e.g. renaming a category) does 4 DB round trips just for auth/authz, before the route's own logic runs at all. Each round trip to Supabase over the public internet is real network latency (commonly 20-100ms+ depending on region/pooler), and this compounds directly with the cold-start problem above - a cold Render instance's first gated request pays cold-start latency *plus* 4 sequential round trips.

- **Fixes, in order of effort:**

1. **Collapse the 3 permission queries into 1** - a single query with a LEFT JOIN across role_permissions and user_permission_overrides, applying the override logic in SQL (or in Python from one combined result set) instead of 3 separate round trips. Purely mechanical, no behavior change.

Feasibility: straightforward. Impact: cuts 3 round trips to 1 on every gated request.

2. **Cache revocation checks in memory** - since revocations are rare (only on logout/admin action), cache the revoked-jti set (or a "not revoked" positive cache with short TTL) in the process, invalidated on write. **Feasibility: moderate - needs care around multi-worker gunicorn processes not sharing memory (would need each worker to independently poll/refresh, which is fine since the set is small). Impact: removes 1 DB round trip from every authenticated request, not just gated ones - this is the higher-leverage of the two since it's checked on every request app-wide.**

3. **Cache a user's resolved role+permissions for the lifetime of a request** - trivial, avoids nothing right now since it's already only computed once per request, but worth noting if any route ever calls it twice.

3. Backend - the categorization pipeline (the "stalls at 100%" one)

What's implemented, accurately:

- Cache tiers (exact -> merchant -> similarity) are separate HTTP round trips so the UI updates progressively.
- LLM batches bundle **up to 200 descriptions into one Gemini call** (not one call per transaction) - genuinely good, this is the single biggest reason it's usable at all.
- temperature: 0, thinking_budget: 0 - deliberately tuned for speed/determinism over the model "thinking more."
- Per-item retry (only the specific descriptions Gemini answered invalidly get retried, not the whole

batch), with a model-fallback chain (primary model, then fallbacks) rather than hammering one overloaded model.

- Cache/merchant writes at the end use `execute_values` (batched multi-row INSERT), not a naive per-row loop.
- A batch-level failure doesn't abort the whole run - only that batch and unattempted ones get marked failed/retryable.

What digging into ``orchestrator.py`` found:

- **Batches run strictly sequentially** - a plain for `batch_index, batch in enumerate(batches)` loop, each blocking on its own `client.models.generate_content` call before the next batch starts. For a large first-time upload with, say, 5 batches of 200 unrecognized descriptions each, and each Gemini call taking (very plausibly) 5-20s for a 200-item structured JSON response, that's 25-100 seconds of pure sequential wall-clock time with nothing else happening concurrently.
- **Cache saves happen once, at the very end** - `global_cache.save()` and `personal_cache.save()` (plus `add_merchants_batch`) only run after every batch has finished. This is almost certainly the exact "stops for a bit even at 100%" symptom: the frontend's progress indicator can reach 100% of *batches processed* while the backend is still doing a final batched write of everything learned across the whole run, before it can even respond.
- **Fix 1 - parallelize batches.** Batches are largely independent (each processes disjoint descriptions), so 2-4 could run concurrently via a thread pool (Gemini calls are I/O-bound, so Python's GIL isn't the blocker) or `asyncio` if the client supports it. The complication: `category_by_description`, `global_cache/personal_cache`, and `personal_resolved` are shared mutable state written by every batch - concurrent batches would need either a lock around those writes (cheap, since the writes themselves are fast, only the network call is slow) or to accumulate each batch's results independently and merge after. *Feasibility: real but non-trivial - needs careful handling of shared state, and Gemini API rate limits need checking (running 4x concurrent requests could hit per-minute quota faster). Impact: potentially cuts total LLM-tier wall-clock roughly proportional to concurrency (e.g. ~3-4x with 4 concurrent batches), which is the single largest lever available for "categorizing takes a while."*
- **Fix 2 - save incrementally, or in the background after responding.** Either write each batch's cache updates as that batch completes (small, frequent batched writes instead of one big one at the end), or fire the final response back to the client immediately and let the merchant/cache writes finish asynchronously (since they're not needed for the transactions the user is about to see - those already have their categories in memory). *Feasibility: straightforward for "save per batch"; the "respond then finish writing" approach needs a background task mechanism (a thread, or a task queue) since Flask's request/response cycle doesn't naturally support "respond now, keep working after." Impact: removes the visible post-100% stall specifically, which is exactly the symptom described - doesn't reduce total processing time, but removes the part of it that's currently invisible/confusing to the user.*
- **Fix 3 - stream LLM batch progress like the cache tiers already do.** Right now the frontend waits for the entire LLM tier's response in one shot; extending the existing "one tier, one round trip, apply as you go" pattern to LLM batches (e.g. one round trip per batch, or Server-Sent Events/polling for progress)

would make the **existing** total time feel much better even before parallelizing it. **Feasibility: moderate* - reshapes the request/response contract for this endpoint. Impact: perceived-speed win, not an actual speed win, but often matters just as much for a bar that currently "hangs at 100%."*

4. Backend - database schema & query design

What's implemented well: `idx_transactions_user_id` exists specifically because "every query filters by `user_id`" (the comment says so directly) - a real, deliberate, documented indexing decision. `idx_category_records_lookup` is a sensible composite index for the cache-tier exact-match lookup pattern.

What digging into `charts.py` and `schema.sql` found:

- `txn_date` is stored as `TEXT`, not a real `DATE` type. The charts-summary query extracts year/month via `SUBSTRING(txn_date FROM 7 FOR 4)` and validates the format via a **regex match on every row** (`txn_date ~ '^d{2}/d{2}/d{4}$'`), then `GROUP BY` on those computed substring expressions. None of this can use an index - Postgres has to regex-match and substring-parse every one of a user's transaction rows, every single time charts load, rather than doing an indexed range scan on a real date column.
 - This is invisible at small scale (a few hundred/thousand rows per user, sub-millisecond either way) but is a genuine scaling problem, and it's also just wasted CPU per request regardless of scale.
 - **Fix:** migrate `txn_date` to a real `DATE` column (a one-time backfill parsing the existing `DD/MM/YYYY` text), use `EXTRACT(YEAR FROM txn_date)/EXTRACT(MONTH FROM txn_date)` instead of substring/regex, and add a composite index on `(user_id, txn_date)`. **Feasibility: moderate* - needs a real migration (no migration framework exists currently, so this would be a manual one-off script, consistent with how schema changes already happen here), and every other place `txn_date` is read/written/compared as text needs auditing (upload parsing, dedup key, transaction list sorting) since this touches a core column. Impact: currently low-to-moderate given likely data volumes, but this is the one fix on this whole list that actually changes the **scaling curve** rather than shaving a constant amount of latency - worth doing before transaction history grows meaningfully per user, not after it becomes a felt problem.*
-

5. Web frontend

What's implemented: client-side fetch timeouts tied to backend worker limits (with a distinct, generous 110s timeout specifically for categorization requests - direct evidence the team already knows this can be slow); server-side aggregation for charts (bounded response size); virtualized transaction list (`@tanstack/react-virtual`); the manual-review batching work from an earlier session.

What digging into `App.jsx` and `AppContext.jsx` found:

- **Zero code-splitting.** Every screen (Home, Charts, Contents, Dashboard, Login, Signup) is imported directly at the top of `App.jsx` - no `React.lazy/Suspense` anywhere, and `vite.config.js` has no manual chunking configured. The entire app, including recharts and every chart sub-component, ships in one JS bundle that has to load and parse before the login screen can even render.

- **Fix:** wrap each route's screen in `React.lazy(() => import(...))` with a `Suspense` boundary at the router level. *Feasibility: low effort, standard React pattern, no architectural change needed. Impact: meaningfully smaller initial bundle/faster first paint, especially valuable given recharts is a non-trivial dependency that a login screen doesn't need at all.*

- **`AppContext`'s Provider value is a fresh object literal on every render, not memoized.** `<AppContext.Provider value={{ ...everything... }}>` with no `useMemo` around that object means **every** component calling `useApp()` re-renders whenever **any** piece of app-wide state changes, anywhere - toggling one chart filter checkbox can cascade re-renders through every mounted consumer of the context, not just the chart.

- **Fix:** wrap the provider value in `useMemo` with the correct dependency array, and/or split the one large context into a few smaller ones by concern (auth/role, transactions, chart-filter state) so a change in one doesn't ripple through consumers of the others. *Feasibility: the `useMemo` wrap is a small, safe change; splitting into multiple contexts is a larger refactor touching every `useApp()` call site. Impact: the `useMemo` fix alone is a classic, well-documented React performance fix - real, client-side-only jank reduction, completely independent of any network/backend latency.*

6. Native mobile app

Same backend-latency exposure as the web app (identical API calls). App-specific findings:

- Drag-to-reorder in the `FilterPane.js` component uses `PanResponder` with state updates on every move event, rather than `react-native-reanimated` worklets (already a dependency) which run on the UI thread without bridging through JS per frame. *Feasibility: a rewrite of that one component's gesture handling. Impact: negligible at small category-list sizes (which is the actual current scale); would matter if someone had a very large category list, which isn't really this app's use case.*
 - No lazy-loading equivalent applies the same way in RN (Metro bundles differently than Vite), so the code-splitting finding above is web-specific.
-

Ranked by effort vs. impact on the reported symptoms

1. **Ping Render, not just Supabase** - smallest effort, directly targets "login/signup take a sec."
2. **Memoize the `AppContext` provider value** - small effort, real client-side snappiness win everywhere in the app, not just one screen.

3. Parallelize LLM batches + save cache incrementally - bigger effort, but this is the actual mechanism behind "categorizing stops for a bit even at 100%," and is the only fix that changes total categorization wall-clock time rather than just hiding it.

4. Collapse the 3 permission queries into 1 - small-to-moderate effort, shaves latency off every gated admin action.

5. Code-split the web bundle - small-to-moderate effort, improves first load specifically.

6. `txn\_date` -> real `DATE` column - the most invasive, and the one with the least *currently felt* impact - but it's the one genuine scaling-curve fix on this list, worth scheduling before it becomes a real complaint rather than after.